SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I P.Sameera Banu (192372172) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Sameera Banu

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTIONS AND ANSWERS:

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

ANSWER:

Servlet-Based Student Registration System

1. Architecture of a Servlet-Based Application

A servlet-based web application generally follows a **client–server architecture**:

```
Student / Browser
      |
      | HTTP Request (GET / POST)
v
   Web Server
      |
      v
Servlet Container (Tomcat)
      |
      v
 StudentRegistrationServlet
      |
      v
 Business Logic / JDBC
      |
      v
  Database
      |
      | HTTP Response
      v
   Browser
```

Main components

1. **Client/Browser** – Displays the registration form and sends HTTP requests.
2. **Web Server** – Receives HTTP requests.
3. **Servlet Container** – For example, Apache Tomcat. It manages servlet creation, execution, sessions, and destruction.
4. **Servlet** – Java class that processes the request and generates the response.
5. **Database** – Stores student information if persistent storage is required.

A typical project structure is:

```
StudentRegistration/
├── src/
│   └── StudentRegistrationServlet.java
├── WebContent/
```

```
│   └── registration.html
└── WEB-INF/
└── web.xml
```

2. HTML Registration Form

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>

<h2>Student Registration</h2>

<form action="register" method="post">
Name:
    <input type="text" name="name"><br><br>

    Email:
    <input type="email" name="email"><br><br>

    Course:
    <input type="text" name="course"><br><br>

    <input type="submit" value="Register">
</form>

</body>
</html>
```

3. Servlet Implementation Using GET and POST

```
import java.io.IOException; import
java.io.PrintWriter;
import jakarta.servlet.ServletException; import
jakarta.servlet.annotation.WebServlet; import
jakarta.servlet.http.HttpServlet; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.servlet.http.HttpServletResponse;

@WebServlet("/register")
public class StudentRegistrationServlet extends HttpServlet {

    @Override
    public void doGet(HttpServletRequest request,
HttpServletResponse response)             throws
ServletException, IOException {         String name
= request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h2>Registration Details (GET)</h2>");
out.println("Name:       "       +       name       +       "<br>");
out.println("Email: " + email + "<br>");
        out.println("Course: " + course + "<br>");
```

```
    }

    @Override     public void
doPost(HttpServletRequest request,
HttpServletResponse response)             throws
ServletException, IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h2>Registration Successful</h2>");
out.println("Name:      "      +      name      +      "<br>");
out.println("Email:      "      +      email      +      "<br>");
out.println("Course: " + course + "<br>");
    }
}
```

doGet() handles HTTP GET requests, while doPost() handles HTTP POST requests.

4. Servlet Life Cycle

The servlet container controls the servlet's life cycle. a)

init()

Called **once**, when the servlet is first created.

```
@Override
public void init() throws ServletException {
    System.out.println("Servlet initialized");
}
```

It is normally used for initialization tasks such as creating resources or loading configuration. b)

service()

Called for each incoming request.

```
HTTP Request
     |
     v
 service()
     |
     +---- GET  ---> doGet()
     |
     +---- POST ---> doPost()
```

The HttpServlet implementation of service() determines the HTTP method and dispatches the request to methods such as doGet() and doPost().

c) destroy ()

Called once before the servlet is removed from service.

```
@Override
public void destroy () {
   System.out.println("Servlet destroyed"); }
```

It is used to release resources such as database connections or files.

Life-cycle sequence

```
Servlet class loaded
      |
      v
  Constructor
      |
v
init()
|
      v
service()
      |
      v
 doGet()/doPost()
      |
      |  (multiple requests)
v     destroy()
```

6. GET vs POST

Feature	GET	POST
Data location	URL/query string	Request body
Visibility	Data appears in URL	Data not normally displayed in URL
Data size	More limited	Better for larger data
Bookmarking	Possible	Generally not appropriate
Sensitive data	Not suitable	Preferred
Typical use	Search/retrieval	Form submission/update

Appropriate method for sensitive data

POST is more appropriate than GET for submitting sensitive information because GET places parameters in the URL, where they may appear in browser history, logs, bookmarks, and other intermediaries.

However, **POST by itself does not encrypt data**. Sensitive information should be transmitted using **HTTPS/TLS**, with proper authentication, authorization, validation, and secure session management.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using Http Session and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

ANSWER:

Session Management Using Http Session and Cookies

When a user logs in, the web application needs to recognize that user on subsequent requests. HTTP itself is stateless, so **session tracking** mechanisms are required.

1. Session Tracking

Without session tracking:

Request 1 → Login
Request 2 → Which user? Request
3 → Which user?

The server does not inherently know that all three requests belong to the same user.

With session tracking:

Browser
  |
  | Login
v Servlet
  |
  | Create HttpSession
  v
Session ID = ABC123
  |
  v
Browser stores session ID
  |
  | Session ID sent with future requests
v Servlet
  |
  v
Retrieve session/user information

The most common approach with HttpSession uses a session identifier, typically carried by a cookie such as JSESSIONID.

2. Login Servlet Using HttpSession

```
import java.io.IOException; import
jakarta.servlet.ServletException; import
jakarta.servlet.annotation.WebServlet; import
jakarta.servlet.http.HttpServlet; import
jakarta.servlet.http.HttpServletRequest; import
jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

   @Override     protected void
doPost(HttpServletRequest request,
HttpServletResponse response)            throws
ServletException, IOException {

       String username = request.getParameter("username");
       String password = request.getParameter("password");

       // Example authentication
       if ("student".equals(username) &&
          "12345".equals(password)) {

          HttpSession session = request.getSession();

          session.setAttribute("username", username);

          response.sendRedirect("home");
       } else {
          response.getWriter().println("Invalid username or password");
       }
   }
}
```

In a real application, credentials should be checked against a database using securely stored password hashes rather than hard-coded values.

3. Retrieving Session Data

A servlet on another page can retrieve the authenticated user's information:

```
import java.io.IOException;
import jakarta.servlet.annotation.WebServlet; import
jakarta.servlet.http.*;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

   @Override     protected void
doGet(HttpServletRequest request,
HttpServletResponse response)            throws
IOException {

      HttpSession session = request.getSession(false);

      if (session == null ||
         session.getAttribute("username") == null) {

         response.sendRedirect("login.html");
return;
      }

      String username =
            (String) session.getAttribute("username");

      response.setContentType("text/html");

      response.getWriter().println(
         "<h2>Welcome, " + username + "</h2>"
      );
   }
}
```

`getSession(false)` returns the existing session without creating a new one.

4. Logout

The session should be invalidated when the user logs out.

```
import java.io.IOException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

   @Override     protected void
doGet(HttpServletRequest request,
HttpServletResponse response)            throws
IOException {

      HttpSession session = request.getSession(false);

      if (session != null) {
         session.invalidate();
      }

      response.sendRedirect("login.html");
   }
}
```

5. Cookies in Servlet Applications

A cookie is a small piece of data stored by the browser and sent with subsequent requests to the appropriate server.

Creating a cookie

```
Cookie userCookie =
      new Cookie("username", username);

userCookie.setMaxAge(60 * 60);

response.addCookie(userCookie);
```

Reading a cookie

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {
   for (Cookie cookie : cookies) {        if
("username".equals(cookie.getName())) {
         String username = cookie.getValue();
         System.out.println(username);
```

```
    }
  }
}
```

For authentication/session identifiers, cookies should generally be configured with appropriate security attributes such as **Secure**, **HttpOnly**, and an appropriate **SameSite** policy.

6. Session-Based Tracking vs Cookies

Feature	HttpSession	Cookies
Stored primarily	Server	Browser/client
Data size	Can hold more server-side data	Small
Security	Sensitive data can remain server-side	Cookie contents are client-visible
Identification	Session ID identifies session	Cookie identifies/stores data
Lifetime	Controlled by server/session settings	Controlled by cookie expiry
Common use	Login/session state	Preferences, identifiers, tracking

An important distinction is that **HttpSession and cookies are not mutually exclusive**. A servlet container commonly uses a cookie containing the **session ID** to allow the server to associate a request with its server-side HttpSession.

Recommended approach

For an authenticated website:

```
User Login
  ↓
Authenticate user
  ↓
Create HttpSession
  ↓
Store user information on server
  ↓
Session ID sent to browser as JSESSIONID
  ↓
Browser sends session ID on subsequent requests
  ↓
Server retrieves HttpSession
  ↓
User remains authenticated
```

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

ANSWER:

JDBC-Based Product Application

1. JDBC Architecture

JDBC (Java Database Connectivity) is a standard Java API for communicating with relational databases.

The architecture can be represented as:

```
Java Web Application
      |
v
    JDBC API
      |
      v
 JDBC Driver Manager
      |
      v
 JDBC Driver
      |
      v
    Database
```

Important JDBC components

- **DriverManager** – Manages JDBC drivers and helps establish connections.
- **Connection** – Represents a connection between the Java application and database.
- **Statement / PreparedStatement** – Executes SQL statements.
- **ResultSet** – Contains data returned by a SELECT query. **SQLException** – Handles database-related errors.

2. Database Table

For example:

```
CREATE TABLE product (
id INT PRIMARY KEY,
name VARCHAR(100),
price DECIMAL(10,2),
   quantity INT
);
```

3. JDBC Steps

The general process is:

1. Load/register JDBC driver
   ↓
2. Establish database connection
   ↓
3. Create PreparedStatement
   ↓
4. Execute SQL
   ↓
5. Process ResultSet
   ↓
6. Close resources

Modern JDBC drivers can usually be automatically discovered, so explicitly calling Class.forName() is often unnecessary.

4. Java JDBC Implementation

```
import java.sql.*;

public class ProductDAO {

   private static final String URL =
      "jdbc:mysql://localhost:3306/ecommerce";

   private static final String USER = "root";
   private static final String PASSWORD = "password";

         // Insert product          public void
insertProduct(int      id,      String      name,
double price, int quantity) {

      String sql =
         "INSERT INTO product " +
         "(id, name, price, quantity) VALUES (?, ?, ?, ?)";

      try (Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);

          PreparedStatement ps =
con.prepareStatement(sql)) {

                ps.setInt(1,  id);
ps.setString(2,            name);
ps.setDouble(3, price);
         ps.setInt(4, quantity);

         int rows = ps.executeUpdate();

         System.out.println(
            rows + " product inserted successfully.");

      } catch (SQLException e) {
         System.err.println(
            "Database error: " + e.getMessage());
```

```
        }
    }

    // Retrieve products
public void getProducts() {

        String sql =
            "SELECT id, name, price, quantity FROM product";

        try (Connection con =
                DriverManager.getConnection(URL, USER, PASSWORD);

             PreparedStatement ps =
                con.prepareStatement(sql);

             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                int id = rs.getInt("id");
                String name = rs.getString("name");
double price = rs.getDouble("price");
                int quantity = rs.getInt("quantity");

                System.out.println(
id + " " + name + " " +
price + " " + quantity);
            }

        } catch (SQLException e) {
            System.err.println(
                "Database error: " + e.getMessage());
        }
    }
}
```

5. Why PreparedStatement?

Instead of constructing SQL like:

```
String sql =
    "INSERT INTO product VALUES (" +
    id + ", '" + name + "', " +
price + ", " + quantity + ")";
```

we use:

```
String sql =
    "INSERT INTO product VALUES (?, ?, ?, ?)";
```

```
PreparedStatement ps = con.prepareStatement(sql);
```

and then:

```
ps.setInt(1,            id);
ps.setString(2,   name);
ps.setDouble(3, price);
ps.setInt(4, quantity);
```

PreparedStatement is preferable because it:

- Helps prevent **SQL injection**.
- Handles parameter values properly.
- Makes SQL easier to maintain.
- Can provide performance benefits for repeated statements.

6. JDBC Application in a Web Architecture

For an e-commerce web application, a better architecture is:

```
Browser
  |
  | HTTP Request
  v
Servlet / Controller
  |
  v
DAO (Data Access Object)
  |
  | JDBC
  v
Database
  |
  | ResultSet
  v
DAO
  |
  v Servlet
  |
  | HTTP Response
  v
Browser
```

For example:

```
@WebServlet("/products")
public class ProductServlet extends HttpServlet {

   private ProductDAO dao = new ProductDAO();

   @Override     protected void
doGet(HttpServletRequest request,
HttpServletResponse response)            throws
IOException {

       dao.getProducts();

       response.setContentType("text/html");
       response.getWriter().println(
          "<h2>Products retrieved successfully</h2>"
       );
   }
}
```

In a production application, the DAO would normally return product objects to the servlet rather than simply printing them to the server console.

7. Exception Handling

JDBC operations can produce SQLException, so database operations should be handled appropriately.

Using **try-with-resources** is recommended:

```
try (Connection con = ...;
PreparedStatement ps = ...;
    ResultSet rs = ...) {

   // Database operations

} catch (SQLException e) {
   // Handle database error
}
```

The try-with-resources statement automatically closes the JDBC resources, even when an exception occurs.

For a real application, database credentials should not be hard-coded. They should be kept in secure configuration/environment settings, and applications should generally use a **connection pool/DataSource** rather than creating a new database connection for every request.